# Chapter 1:

## LEARNINGS DURING THE INTERNSHIP

My internship at GRID R&D has been a rich tapestry of learning experiences. From hands-on projects to collaborative tasks, each opportunity has added a layer of knowledge and skill to my professional growth. In this section, I'll explore the key learnings that have not only deepened my understanding of the industry but have also equipped me with valuable tools for the future.

### 1.1. Preparation of Project Initial Documentation (PID):

The Project Initial Documentation (PID) serves as the foundational blueprint for any project, outlining its scope, objectives, resources, timelines, and risks. It typically includes the following key components:

1. Project Overview: A brief description of the project, its purpose, and the desired outcomes. This section sets the context for the entire document.

2. Objectives: Clear, measurable goals that the project aims to achieve. These objectives should be specific, achievable, relevant, and time-bound.

3. Scope: Defines the boundaries of the project, detailing what is included and what is not. It helps prevent scope creep and ensures stakeholders have a clear understanding of what to expect.

4. Stakeholders: Identification of individuals or groups with an interest in the project, along with their roles and responsibilities.

5. Deliverables: Tangible outputs or outcomes that the project will produce. Each deliverable should be defined and quantifiable.

6. Timeline: A timeline outlining key milestones and deadlines for the project, including major phases or stages.

7. Resources: Allocation of necessary resources such as budget, personnel, equipment, and materials required for the project's execution.

8. Risks and Mitigation Strategies: Identification of potential risks that may impact the project's success, along with strategies to mitigate or manage these risks.

The PID serves as a reference point throughout the project lifecycle, guiding decision-making and ensuring alignment with organizational goals and stakeholder expectations. It provides a structured framework for planning, executing, and monitoring the project, ultimately enhancing its chances of success.

## 1.2. Preparation of Software Requirement Specification (SRS):

The Software Requirements Specification (SRS) is a comprehensive document that outlines the functional and non-functional requirements of a software system to be developed. It serves as a communication bridge between stakeholders, including clients, users, and developers, ensuring a common understanding of the software's features and capabilities. The SRS typically includes the following components:1. Introduction: Provides an overview of the document, including its purpose, scope, and intended audience.

2. General Description: Offers a high-level description of the software system, including its objectives, functionalities, and constraints.

3. Specific Requirements: Details the specific features and functionalities of the software, often organized into subsections such as functional requirements, interface requirements, performance requirements, etc.

4. External Interface Requirements: Describes the interfaces between the software system and external entities, such as users, hardware devices, or other software systems.

5. Functional Requirements: Defines the specific functions that the software must perform, typically in the form of use cases, user stories, or functional specifications.

6. Non-Functional Requirements: Specifies the quality attributes or constraints of the software system, such as performance, security, usability, scalability, and reliability.

7. System Models: Includes various models such as data flow diagrams, entity-relationship diagrams, or state-transition diagrams to illustrate the system's structure and behavior.

8. Appendices: May include additional information such as glossary of terms, references, or supplementary documents.

The SRS serves as a foundation for software development, guiding the design, implementation, and testing phases. It helps stakeholders to align their expectations and provides a basis for evaluating the software's compliance with requirements throughout the development lifecycle. Additionally, the SRS can serve as a contractual agreement between the client and the development team, ensuring that the delivered software meets the agreed-upon specifications.

## 1.3. Studying Datasheet:

During my internship at GRID R&D, deciphering the intricate language of datasheets became a cornerstone of my learning experience. These documents, often perceived as dry technical manuals, revealed themselves as portals to understanding the nuances of the project's building blocks. From meticulously analyzing the voltage and current ratings of the Arduino Pro Mini to dissecting the operating temperatures of the DC motor pump, each datasheet unveiled the unique capabilities and limitations of each component. This newfound knowledge proved invaluable in selecting compatible parts, designing efficient circuits, and troubleshooting issues that arose during the development of our automated bathroom fumigator.

Beyond the basic specifications, datasheets often held hidden gems. Application notes for the single nozzle fogger offered insights into its optimal spraying patterns, while timing diagrams for the 555 timer circuit provided crucial guidance for pulse-width modulation control. Learning to interpret these additional sections empowered me to push the boundaries of the components and explore innovative solutions, ultimately enhancing the project's functionality and efficiency.

The ability to effectively analyze and utilize datasheets has transcended the specific context of this internship. It has equipped me with a valuable skill that I can readily apply to future projects, fostering a sense of intellectual curiosity and a commitment to continuous learning. Whether delving into the intricacies of a new sensor or navigating the complexities of a different microcontroller, I am now better equipped to understand the underlying principles and make informed decisions, thanks to the invaluable lessons learned from the language of datasheets.

## 1.4. Teamwork:

While technical skills held immense value during my internship at GRID R&D, the project's success hinged heavily on the power of teamwork. Our team, composed of individuals from diverse backgrounds and expertise, fostered an environment of collaboration and mutual learning.

### Synergy of Ideas:

From brainstorming sessions to troubleshooting hurdles, open communication and active participation were key. Each member brought unique perspectives and skillsets to the table, enriching discussions and leading to innovative solutions. Whether it was the mechanical engineer's suggestion for optimizing the fumigation nozzle or the software developer's insightful code review, diverse viewpoints

proved invaluable in overcoming challenges and refining the project.

**Shared Ownership and Support:**

A strong sense of shared ownership fostered a collaborative spirit. We celebrated individual achievements while recognizing the collective effort behind them. This supportive environment encouraged open communication, allowing team members to ask questions, seek help, and offer assistance without hesitation. Whether it was lending a hand with complex coding tasks or providing feedback on circuit designs, the collaborative spirit ensured everyone felt valued and contributed meaningfully.

**Beyond the Project:**

The teamwork experience extended beyond the technical aspects. We celebrated successes together, supported each other through setbacks, and fostered a sense of camaraderie that transcended the project's timeline. This positive and collaborative environment fostered personal growth, allowing me to develop communication skills, learn from diverse perspectives, and appreciate the power of teamwork in achieving common goals.

**Lasting Impact:**

The lessons learned through collaborative work during this internship have proven invaluable. I have gained a strong appreciation for the power of diverse perspectives, the importance of open communication, and the supportive environment that teamwork fosters. These skills will undoubtedly benefit me in future endeavors, regardless of the project or team I encounter.

## 1.5. Leadership:

Leading the team for the bathroom fumigation project at GRID R&D honed my leadership skills. I delegated tasks, facilitated communication, and made key decisions, fostering accountability and strong decision-making. The project also demanded adaptability and a growth mindset. I embraced challenges by seeking solutions, exploring alternatives, and incorporating feedback, solidifying the importance of flexibility and continuous learning – skills valuable for any future endeavor.

# CHAPTER 2

# WEEKLY INTERNSHIP ACTIVITIES

## 2.1 Week - 1: Project Initial Stage

- **Team Formation:** A team of 4 members was formed to work on the project.
- A team leader was appointed to coordinate and manage the project.
- **Project Phases:** The Project was divided into different phases to ensure a structured development.
- **System Components:** PIR sensor for motion detection, Tank to store the sanitation liquid, DC motor to pump the sanitation liquid from tank to fogger, Fogger Nozzle to spray the sanitation liquid.
- **Component Research:** Week 1 activity was completely based on learning about the components and their functionality.
- **Datasheet Analysis:** Datasheets are the main source of information about the components ranging from their operational voltage and current levels to their functionality.

## 2.2 Week -2: Software Simulation

- The first task of week 2 was to get familiar with proteus software and simulate simple circuits to understand the working.
- The subsequent tasks was to build simple circuits using the components to understand their working and timing constraints.
- For Pir sensor a simple LED switch circuit was built using Arduino Pro mini.
- The LED was replaced with motor for further understanding and learning new logical approaches.
- We even understood the working of a 555 timer and built a simple delay timer.

## 2.3 Week -3: Algorithm Development

- The main task of week 3 was to completely implement the working prototype of the bathroom fumigator using Arduino Pro mini microcontroller.
- Based on our learning from the previous weeks, we integrated all the components to the microcontroller.
- Software algorithm for the complete working of the device had to be developed multiple times based on the final result and feedback we attained.
- Multiple algorithms were tested based on different scenarios that might occur.
- The final algorithm had covered all the possible situations and the device was simulated.

## 2.4 Week-4: Analog Circuit Design

- The main task of this week was to design the same circuit using a 555 timer in place of a microcontroller.
- The main aim of this week was to introduce us to the industrial work standards and thought process.
- We had to complete the same task with a much more cost effective and space efficient approach.
- Our first approach to this task was to understand working of a 555 timer.
- We researched about different delay circuits and can be developed using the 555 timer.
- Upon multiple trials using different circuits, we built a logic circuit to have the desired functionality.

# CHAPTER 3

# PROJECT DESCRIPTION DETAILS

## 3.1 Problem Statement

A restroom or washroom is a room typically in a hotel or other commercial building, that contains either a toilet and a sink (or both) that is used by the public either to wash their hands or any other personnel hygiene activities. Such washrooms might contain a lot of bacteria and other infectious viruses which can infect people if not sanitized properly. To overcome such problems fumigation system must be included.

## 3.2 Objectives

- Replace manual hygiene maintenance with an automated system.
- Provide a periodic fumigation process to maintain hygiene.
- Detects the presence of humans to avoid inappropriate fumigation.

## 3.3 Methodology

**Hardware:**

- **Microcontroller:** Choose an Arduino board or a similar microcontroller platform compatible with your needs. Consider factors like available pins, power requirements, and programming experience.
- **DC water pump:** Ensure the pump's voltage and current ratings match your microcontroller's capabilities. Choose a pump suitable for your application's water flow and pressure requirements.
- **PIR sensor:** Select a PIR sensor with adjustable sensitivity and range based on your detection area and desired trigger distance.
- **Relay module (optional):** If your water pump draws more current than your microcontroller can handle directly, use a relay module for safer and more efficient control.
- **Power supply:** Choose a power supply that can provide enough current for both the microcontroller and the pump (if not using a relay).
- **Connecting wires:** Use appropriate gauge wires for your project's current requirements and ensure proper connections.
- **Breadboard or prototyping shield (optional):** Use these for easier prototyping and testing if

desired.

**Software:**

- **Programming language:** Use Arduino IDE or a similar platform suitable for your chosen microcontroller.
- **Libraries:** Depending on your chosen sensor and relay module, you might need specific libraries for their functionalities.

**Methodology:**

**1. Hardware Setup:**

1. **Connect the DC water pump:**
    - If using a relay module:
        - Connect the pump to the relay's output terminals according to its datasheet.
        - Connect the relay's control pin to the microcontroller's digital output pin.
    - If not using a relay:
        - Connect the pump directly to the microcontroller's output pin, ensuring current compatibility.
2. **Connect the PIR sensor:**
    - Connect the sensor's power and ground pins to the microcontroller's corresponding pins.
    - Connect the sensor's output pin to a digital input pin of the microcontroller.
3. **Connect the power supply:**
    - Connect the power supply to the microcontroller and the pump or relay module as needed.
4. **Optional: Test the connections:**
    - Briefly power on the system and manually control the pump using the microcontroller to ensure proper functionality.

**2. Software Setup:**

1. **Include necessary libraries:**
    - If using libraries for sensors or relays, include them in your Arduino sketch.
2. **Define pin assignments:**
    - Declare variables for the digital pins connected to the pump control (relay) and PIR sensor.

3. **Initialize pins:**
    - Set the pump control pin as OUTPUT and the PIR sensor pin as INPUT in the `setup` function.

**3. Main Program Loop:**

1. **Read PIR sensor value:**
    - Within the `loop` function, read the sensor value using `digitalRead(PIR_pin)`.
2. **Motion Detection:**
    - If motion is detected (LOW reading):
        - Check if the pump was previously off:
            - If yes (`previous_state == HIGH`):
                - Turn on the pump:
                    - If using a relay: `digitalWrite(relay_pin, HIGH)`.
                    - If directly controlling: `digitalWrite(pump_pin, HIGH)`.
                - Print a message to the serial monitor (optional).
                - Record the current time as `start_time`.
        - Update the `previous_state` variable with the current reading.
3. **Motion Stopped or Timeout:**
    - If motion is not detected (HIGH reading):
        - Check if the pump was previously on:
            - If yes (`previous_state == LOW`):
                - Turn off the pump:
                    - If using a relay: `digitalWrite(relay_pin, LOW)`.
                    - If directly controlling: `digitalWrite(pump_pin, LOW)`.
                - 
                - Print a message to the serial monitor (optional).
                - Calculate the time since `start_time`.
                - If the elapsed time is less than a desired minimum duration (e.g., 5 seconds):
                    - Restart the timer to avoid turning off immediately after brief motion.
        - Update the `previous_state` variable with the current reading.

4. **Optional Enhancements:**
    - **Adjust sensitivity:** Implement adjustable sensitivity for the PIR sensor using its settings or code manipulation.
    - **Debouncing:** Use debouncing techniques to avoid multiple readings due to sensor noise.
    - **Timed operation:** Allow manual configuration of pump activation duration after motion detection.
        - **Advanced control:** Implement more complex control logic like multiple pump modes or integration with other sensors.

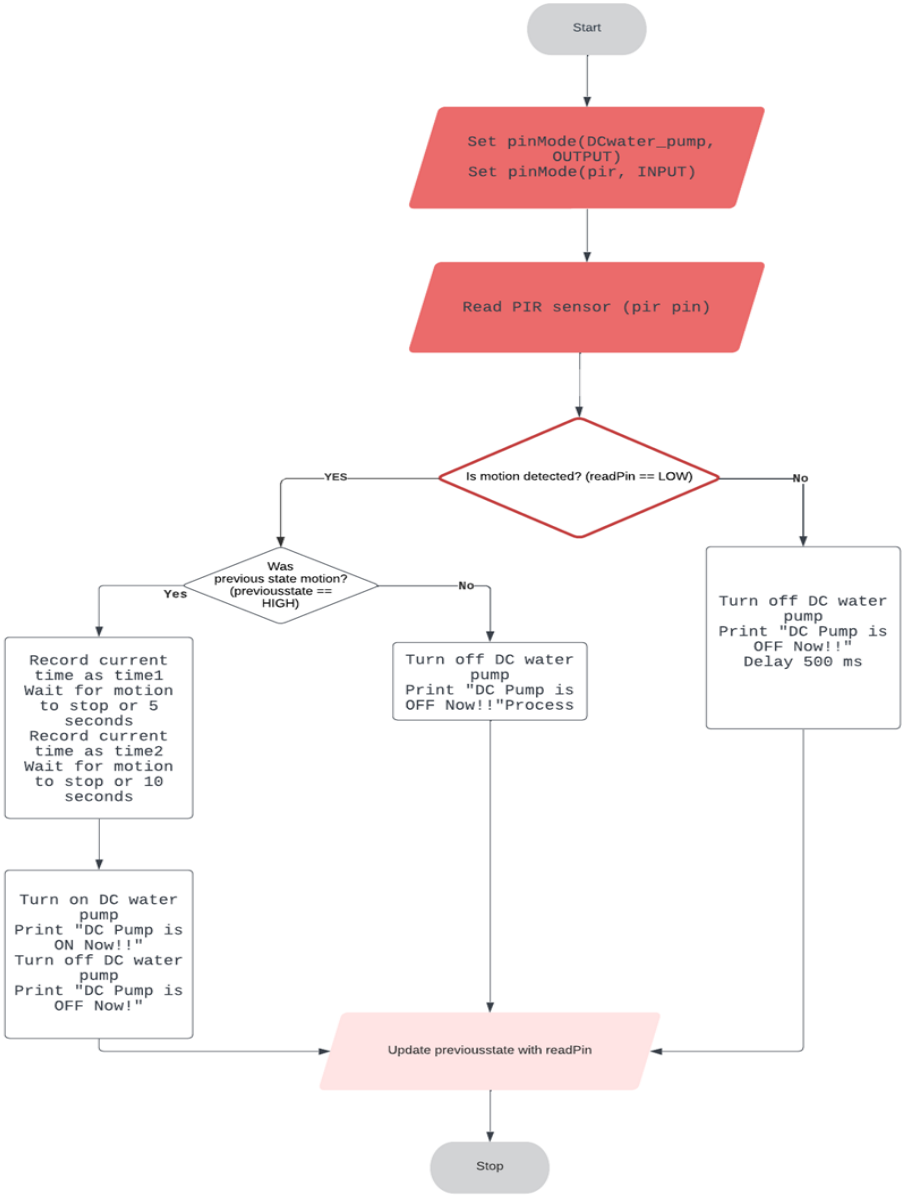



**Fig 4.3.1. Flowchart of the Product**

## 3.4 Hardware and Software Requirements

### Arduino Pro Mini

The Arduino Pro Mini is a microcontroller board based on the ATmega328P.It has 14 digital input/output pins (of which 6 can be used as PWM outputs), 6 analog inputs, an on-board resonator, a reset button, and holes for mounting pin headers. A six pin header can be connected to an FTDI cable or Sparkfun breakout board to provide USB power and communication to the board. The Arduino Pro Mini is intended for semi-permanent installation in objects or exhibitions. The board comes without pre-mounted headers, allowing the use of various types of connectors or direct soldering of wires. The pin layout is compatible with the Arduino Mini.There are two version of the Pro Mini. One runs at 3.3V and 8 MHz, the other at 5V and 16 MHz.The Arduino Pro Mini was designed and is manufactured by SparkFun Electronics.

*Specifications:*

Board Power Supply: 3.35 -12 V (3.3V model) or 5 - 12 V (5V model)

Circuit Operating Voltage: 3.3V or 5V (depending on model)

Digital I/O Pins: 14

PWM Pins: 6

UART: 1

SPI: 1

I2C: 1

Analog Input Pins: 6

External Interrupts: 2

DC Current per I/O Pin: 40mA

Flash Memory: 32KB of which 2 KB used by bootloader

SRAM: 2 KB

EEPROM:1KB

Clock Speed: 8 MHz (3.3V versions) or 16 MHz (5V versions)

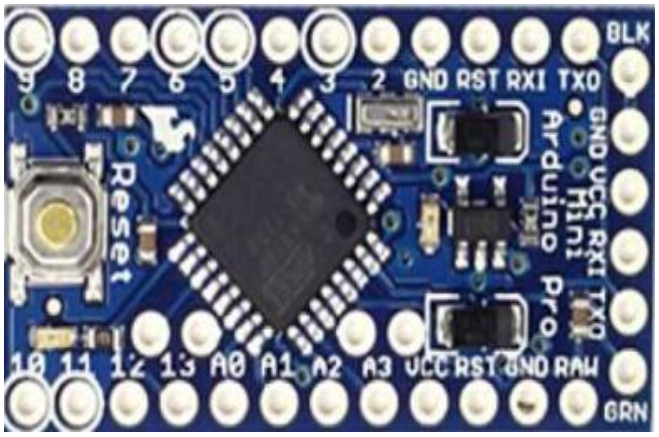



**Fig 4.4.1. Arduino Pro Mini.**

## FTDI Cable

The FTDI cable is a USB to Serial (TTL level) converter which allows for a simple way to connect TTL interface devices to USB. The I/O pins of this FTDI cable are configured to operate at 5V.The FTDI cable is designed around an FT232RQ, which is housed in a USB A connector. The other side of the cable is terminated with a 0.1" pitch, 6-pin connector with the following pinout: RTS, RX, TX, 5V, CTS, GND (RTS is the green cable and GND is black).

This cable has the same pinout and functionality as our FTDI Basic Breakout board; you can use it to program your Arduino Pro, Pro Mini and Lilypad. For use with those boards, align the black and green wires of the FTDI cable with the 'BLK' and 'GRN' labels on the PCB.

There are pros and cons to the FTDI Cable vs the FTDI Basic. The FTDI Basic has great LED indicators, but requires a Mini-B cable. The FTDI Cable is well protected against the elements, but is large and cannot

be embedded into a project as easily. The FTDI Basic uses DTR to cause a hardware reset where the FTDI cable uses the RTS signal.

**Fig 4.4.2. FTDI Cable**



## CP2102

The CP2120 is a highly integrated SPI-to-I2C Bridge Controller with an SPI interface that provides a simple and reliable method for communicating with I2C devices. The CP2120 includes a 4-wire serial peripheral interface (SPI), a serial I2C interface, 256-byte data buffers, an internal oscillator, eight input/output port pins, and one pin configurable as an edge-triggered interrupt source in a compact 4x4 package. No external components other than pull-up resistors on the I2C pins are required. The SPI Master controls the CP2120 across the SPI interface using a command set that governs all CP2120 configuration and operation.




**Fig 4.4.3. CP2120 Chip**

## LED

A light-emitting diode (LED) is a semiconductor device that emits light when current flows through it. Electrons in the semiconductor recombine with electron holes, releasing energy in the form of photons. The color of the light (corresponding to the energy of the photons) is determined by the energy required for electrons to cross the band gap of the semiconductor. White light is obtained by using multiple semiconductors or a layer of light-emitting phosphor on the semiconductor device.

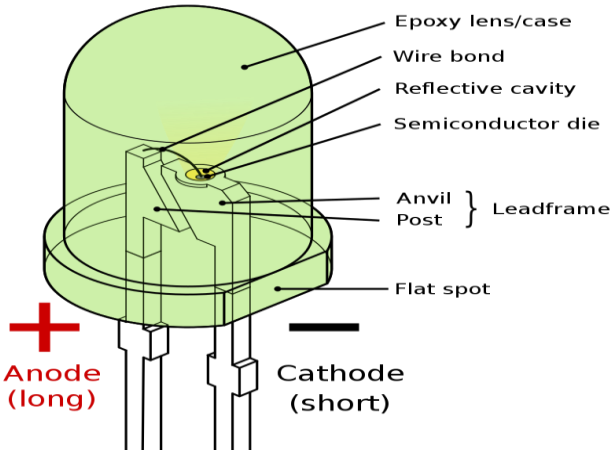



**Fig 4.4.4. LED**

## Jumping Wires

A jump wire (also known as jumper, jumper wire, DuPont wire) is an electrical wire, or group of them in a cable, with a connector or pin at each end (or sometimes without them – simply "tinned"), which is normally used to interconnect the components of a breadboard or other prototype or test circuit, internally or with other equipment or components, without soldering. Individual jump wires are fitted by inserting their "end connectors" into the slots provided in a breadboard, the header connector of a circuit board, or a piece of test equipment.

There are different types of jumper wires

- Solid tips – are used to connect on/with a breadboard or female header connector. The arrangement of the elements and ease of insertion on a breadboard allows for increasing the mounting density of both components and jump wires without fear of short-circuits. The jump wires vary in size and colour to distinguish the different working signals.



**Fig 4.4.5. Jumper Wire**

## Single Nozzle Fogger

A single nozzle fogger is a device used to create fine mist or fog by dispersing liquids into tiny droplets. It typically consists of a nozzle through which a liquid is pressurized and forced to pass. As the liquid exits the nozzle, it is broken down into small droplets, creating a fog-like effect.



.

**Fig 4.4.6. Single Nozzle Fogger**

## DC Motor Pump

A DC motor is an electrical motor that uses direct current (DC) to produce mechanical force. The most common types rely on magnetic forces produced by currents in the coils. Nearly all types of DC motors have some internal mechanism, either electromechanical or electronic, to periodically change the direction of current in part of the motor.DC motors were the first form of motors widely used, as they could be powered from existing direct-current lighting power distribution systems. A DC motor's speed can be controlled over a wide range, using either a variable supply voltage or by changing the strength of current in its field windings



**Fig 4.4.7. DC Water Pump**

## PIR (Passive Infrared) Sensor

A PIR (Passive Infrared) sensor, also known as a PIR motion sensor, is an electronic device that detects changes in infrared radiation within its field of view. It's commonly used to detect the presence of moving objects, particularly humans and animals, by sensing the heat emitted by their bodies.



**Fig 4.4.8. Pir Sensor**

## Resistor

A passive electrical component with two terminals that are used for either limiting or regulating the flow of electric current in electrical circuits. The main purpose of the resistor is to reduce the current flow and to lower the voltage in any particular portion of the circuit. It is made of copper wires which are coiled around a ceramic rod and the outer part of the resistor is coated with an insulating paint. SI unit is **ohm**

Symbol**:**



**Fig 4.4.9. Resistor Symbol**

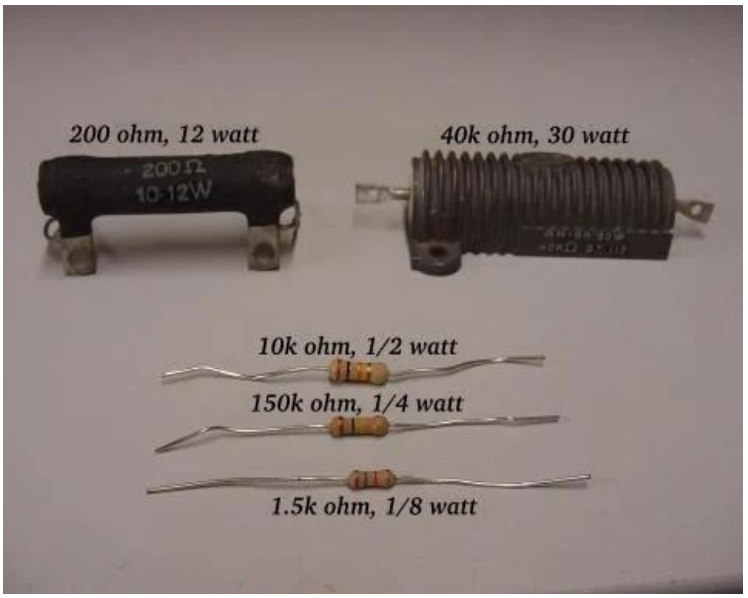



**Fig 4.4.10. Different Resistors.**

## Arduino Software IDE

The Arduino IDE is an open-source software, which is used to write and upload code to the Arduino boards. The IDE application is suitable for different operating systems such as Windows, Mac

OS X, and Linux. It supports the programming languages C and C++. Here, IDE stands for Integrated Development Environment.

The program or code written in the Arduino IDE is often called sketching. We need to connect the Genuino and Arduino board with the IDE to upload the sketch written in the Arduino IDE software. The sketch is saved with the extension '.ino.'

***The Arduino IDE will appear as:***

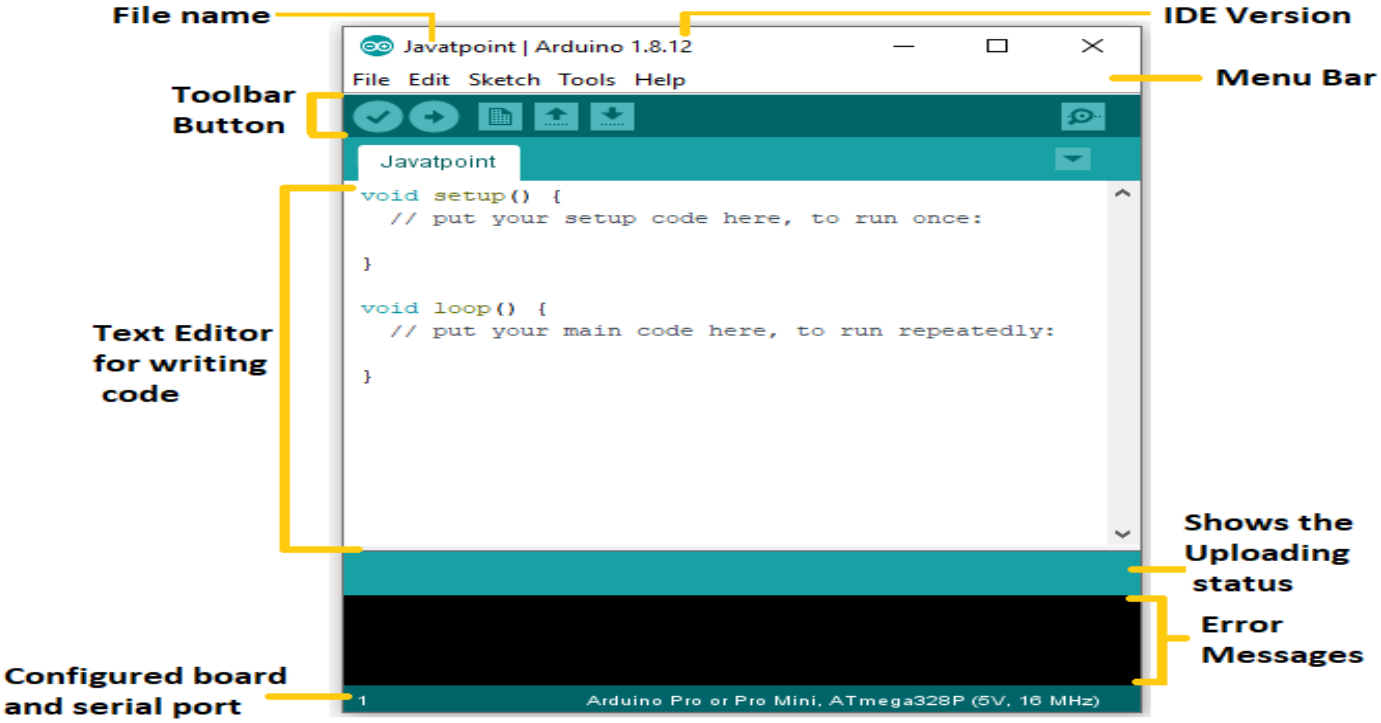



**Fig 4.4.11. Arduino IDE**

## Proteus software

The Proteus Design Suite is a proprietary software tool suite used primarily for electronic design automation. It is a Windows application for schematic capture, Simulation, and Printed Circuit Board(PCB) layout design. It can be found in many configurations, depending on the size of designs being produced and the requirements for microcontroller simulation. All PCB Design products include an autorouter and basic mixed-mode SPICE simulation capabilities.

The software is used mainly by electronic design engineers and technicians to create schematics and electronic prints for manufacturing printed circuit boards(PCBs) and also as a rapid prototyping tool for

R&D. It is also found in Universities across the world, teaching electronics, embedded design, and PCB layout to students. It also has features that allow you to virtually simulate your IoT projects.

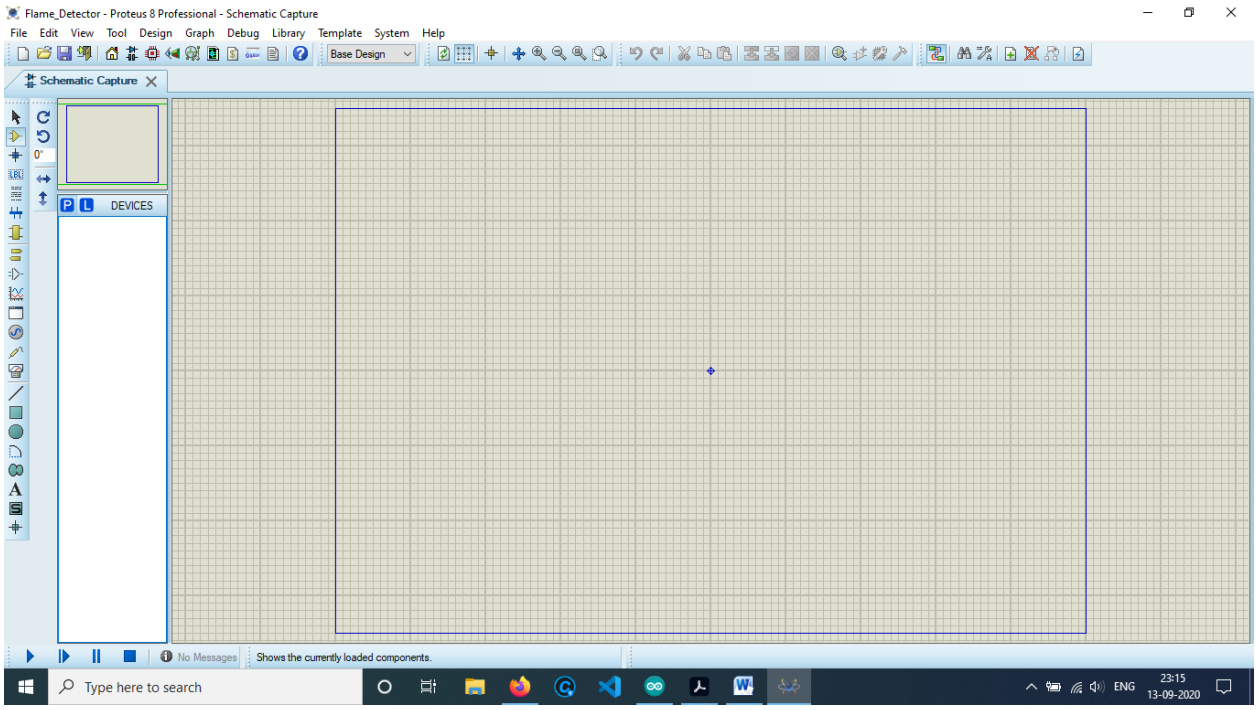



**Fig 4.4.12. Proteus Software**

*Features of Proteus:*

- ISIS Schematic Capture: it is a powerful schematic tool that helps make and simulate electronic circuits easily. It supported the different components like active passive and digital components. We can make mixed-mode circuits with the use of digital and analog components
- ARES PCB Layout: it is a PCB layout tool that helps us to design and manufacture printed circuit boards. It supported different types of PCB components, like footprints, traces, pads, and vias. There is ARES can be used for DRC and ERC checks on PCB boards
- 3D Visualization: Proteus also has a 3D visualization tool that is helpful to view board designs in a 3d way. It is best of check the board layout and find any potential issues
- Library Manager: Proteus also comes with a library manager that helps us to store and manage electronic components
- Simulation Engine: This software also comes with a powerful simulation engine that helps us to simulate the operation of our circuit. It is best to test circuits before making it and debug any issue if your find

- Export Options: it also comes with different options like drill files Gerber files, and BOMs. it helps us to send board design to the manufacturer for manufacturing

## 4.5 Results and Analysis

Figure 4.5.1 shows the circuit diagram of a washroom fumigator using an Arduino Pro Mini Development board. Using the Proteus Simulation Software the algorithm and the circuit were tested for their functionality and necessary changes were made periodically based on the requirements given to us.

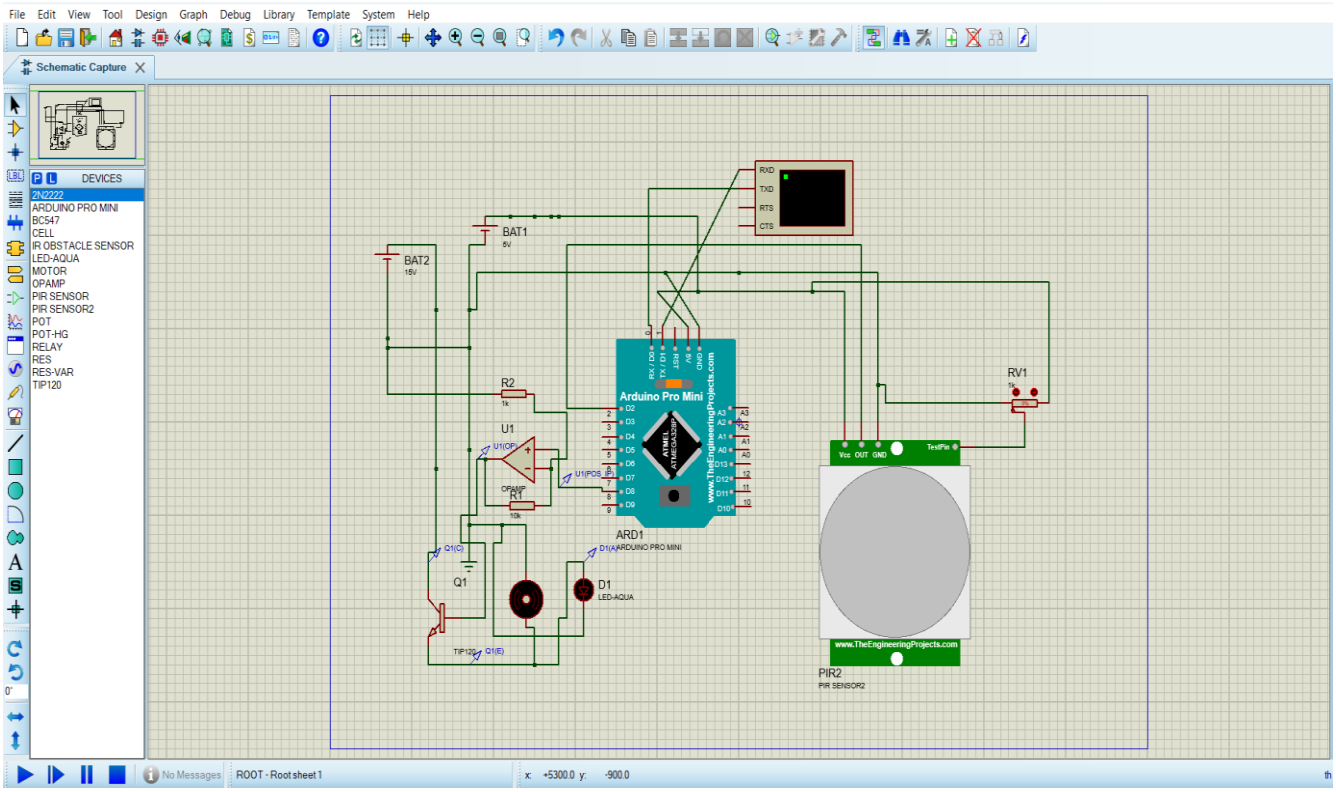



**Fig 4.5.1. Circuit Diagram using Arduino Pro Mini**

Figure 4.5.2 shows the circuit diagram of a washroom fumigator using a 555 delay timer in place of the microcontroller. Proteus software was used to develop the circuit and check the functionality.

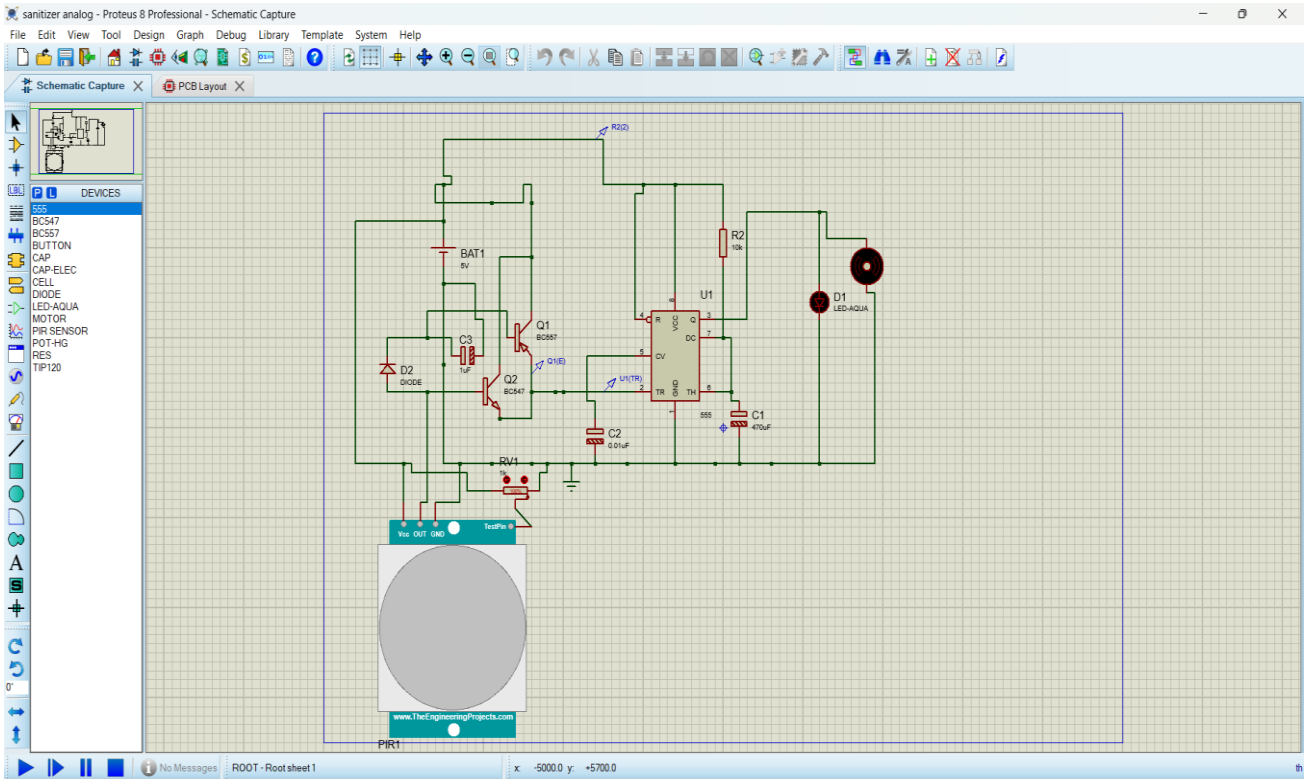



**Fig 4.5.2. Circuit Diagram using 555 timer**

# CHAPTER 4

## REFLECTION ON THE INTERNSHIP

My internship at GRID R&D concluded, leaving a lasting impression of professional growth and personal fulfilment. This journey commenced with a mixture of eagerness and uncertainty, fueled by academic knowledge and a desire for practical application. Over the following months, this initial curiosity evolved into a rewarding experience that transformed me both professionally and personally.

Initially, the intricacies of the automated bathroom fumigation project seemed daunting. However, the supportive team environment and my newfound skill in deciphering datasheets provided a solid foundation. From meticulously programming the Arduino controller to understanding the nuances of sensor integration, each hurdle overcome fueled a thirst for knowledge and a drive to conquer the next challenge.

Beyond technical expertise, the internship fostered valuable soft skills. Teamwork became second nature as we brainstormed solutions, celebrated successes, and supported each other through roadblocks. Communication skills blossomed through collaborative discussions and presentations, while time management became critical as we navigated deadlines and unexpected issues. These skills, honed throughout the project, proved invaluable not only within the team but also in my personal growth.

The internship transcended mere technical accomplishments and skill development. It offered an opportunity to witness the thrill of innovation firsthand, observing how seemingly disparate components could be transformed into a tangible solution with real-world impact. The potential of the fumigation system to enhance hygiene and reduce disease transmission instilled a deeper meaning in the project, motivating me to push my boundaries and contribute meaningfully.

As the internship concludes, I carry with me not just technical knowledge and honed skills, but a newfound confidence and a commitment to continuous learning. The invaluable lessons gleaned from this experience will undoubtedly guide me in my future endeavours, reminding me of the power of collaboration, resilience, and a passion for innovation. This internship wasn't just a professional stepping stone; it was a transformative journey that shaped me into a more well-rounded individual, prepared to tackle future challenges with an open mind and a collaborative spirit.

## Appendix 1: Code of the Project

```
/*
 * Bathroom Fumigation System
 * This program uses a PIR sensor to detect human presence in a bathroom.
 * When a person leaves the bathroom, it activates a DC motor to dispense a sanitizing liquid
 * and turns on an LED to indicate the fumigation process. After a certain delay, it turns off the DC motor and LED.
 *
 * Components:
 * - PIR Sensor
 * - DC Motor
 * - LED
 *
 * Wiring:
 * - PIR Sensor Output to Arduino Digital Pin 2
 * - DC Motor Control Pin to Arduino Digital Pin 8
 * - LED Control Pin to Arduino Digital Pin 8
 *
 * Author:Reshika N
 * created on 26th August 2023
 *
```

```
 * Note: Make sure to connect appropriate power and ground connections for each component.
 */
#define DCwater_pump 8
#define pir 2
unsigned long time1,time2;
int previousstate=0;
void setup() {
  pinMode(DCwater_pump, OUTPUT);
  pinMode(pir,INPUT);
  Serial.begin(9600);
}
void loop() {
  pirsensor();
}
void pirsensor(){
int readPin = digitalRead(pir);
Serial.println(readPin);
if (readPin == LOW && previousstate == HIGH)
{
time1=millis()/1000;
 int sense=0;
```

```
 while(sense==0 && millis()/1000-time1<5){
   sense=digitalRead(pir);
 }
 time2=millis()/1000;
 while(sense==0 && millis()/1000-time2<10){
 sense=digitalRead(pir);
 digitalWrite(DCwater_pump,HIGH);
 Serial.println("DC Pump is ON Now!!");
}
 digitalWrite(DCwater_pump,LOW);
 Serial.println("DC Pump is OFF Now!");
}
else
{
 digitalWrite(DCwater_pump,LOW);
 Serial.println("DC Pump is OFF Now!!");
 delay(500);
}
 previousstate=readPin;
 Serial.println(previousstate);
}
```